

DIGITAL IMAGE PROCESSING (CS 406)

Class Activity 5: Spatial Domain Image Filtering

Programming in Python for Morphology, Smoothing, Gradients, Pyramids & Edge Detection

Instructor: Dr. Jameel Ahmad
Department: Computer Science, SST UMT
Grading Basis: Correctness 50%, Output Quality 20%, Readability 10%, Analysis 20%

Term: Summer 2026
Submission Date: September 23, 2026
Source Notebook: `cas/ca_05/ca_05.ipynb`

Student Name	Reg. No.	Primary Task(s)
Muhammad Umer	F2023266912	Task 1 (Matplotlib + OpenCV), Task 2 (Morphology), Task 3 (Smoothing & Blurring)
Haseeb Ullah	F20232661009	Task 4 (Gradients), Task 5 (Pyramids & Blending), Task 6 (Edge Detection)

Note: CA-5 asks for groups of 4. Our group has 2 active members; each took three of the six tasks so that every task was completed in full and both members can independently explain every task, as required for the in-class viva.

Executive Summary

This report documents a complete, hands-on tour of spatial-domain image filtering in OpenCV: displaying and comparing images with Matplotlib; morphological transformations (erosion, dilation, opening, closing); five smoothing/blurring filters (Gaussian, Median, Bilateral, Box, Mean) compared quantitatively on noise removal and edge preservation; Sobel-based image gradients (magnitude and direction); Gaussian/Laplacian image pyramids used for classic Burt–Adelson-style pyramid blending; and three edge-detection methods (Sobel+threshold, Laplacian of Gaussian, Canny), including Canny applied frame-by-frame to a video stream. All results in this report were generated by executing `cas/ca_05/ca_05.ipynb` end-to-end; every figure below is a real, measured output of that run, not a mock-up, and every numeric claim is backed by a printed statistic from the notebook.

Contents

1	Setup, Environment & Dataset Construction	3
1.1	Dataset	3
2	Task 1 – Matplotlib with OpenCV (Muhammad Umer)	4

3	Task 2 – Morphological Transformations (Muhammad Umer)	6
4	Task 3 – Smoothing and Blurring (Muhammad Umer)	8
5	Task 4 – Image Gradients (Haseeb Ullah)	10
6	Task 5 – Image Pyramids and Blending (Haseeb Ullah)	12
6.1	Gaussian and Laplacian pyramids (4 levels)	12
6.2	Blending apple.jpg and orange.jpg	12
7	Task 6 – Edge Detection (Haseeb Ullah)	14
7.1	Sobel + threshold, Laplacian of Gaussian, and Canny	14
7.2	Canny edge detection on a video stream	15
8	Rubric Self-Assessment	17
9	Summary of Findings	17
10	Conclusion & Individual Contributions	17

1 Setup, Environment & Dataset Construction

The experimentation uses Python 3.13 with `OpenCV (cv2)`, `NumPy`, and `Matplotlib`. Display throughout is done with `matplotlib.pyplot.imshow` rather than `cv2.imshow()`, which opens a blocking native window unsuitable for a notebook meant to be re-run top-to-bottom non-interactively.

1.1 Dataset

The CA-5 brief references generic sample filenames (`out.jpg`, `apple.jpg`, ...). With no internet access available to fetch new stock photos, we built a small, purpose-specific, fully reproducible dataset from photographs already bundled in this repository's course dataset (`../data`) plus a few images constructed synthetically with OpenCV drawing primitives, exactly as for CA-4:

- `out.jpg` – an outdoor scene photo (`data/chp_01_images/scene.jpg`), used for Task 1.
- `apple.jpg` / `orange.jpg` – two synthetic, identically-sized (420×420) shaded fruit images built specifically for the classic Burt–Adelson pyramid-blending demo in Task 5; building them ourselves guarantees both fruits share the exact same size/position needed for a clean half-and-half blend.
- `shapes_noisy.png` – a synthetic binary image (rectangle/circle/triangle) with small bright specks scattered over the background and small dark holes scattered inside the shapes, used for Task 2, so that every morphological operation has a directly visible and countable effect.
- `noisy_parrot.jpg` – `data/chp_02_images/parrot.jpg` with synthetic Gaussian ($\sigma=22$) and salt-and-pepper noise (1% each) added, used for Task 3 – comparing denoising filters is only meaningful on an image that actually contains noise.
- `board.jpg` – a high-contrast chalkboard photo (`data/chp_01_images/board.jpg`), used for Task 4 and Task 6, where strong, well-defined strokes make Sobel/LoG/Canny comparisons easy to read.
- `football.mp4` – an existing video (`data/chp_01_images/football.mp4`), referenced directly (not copied) for Task 6's video-based Canny demo.

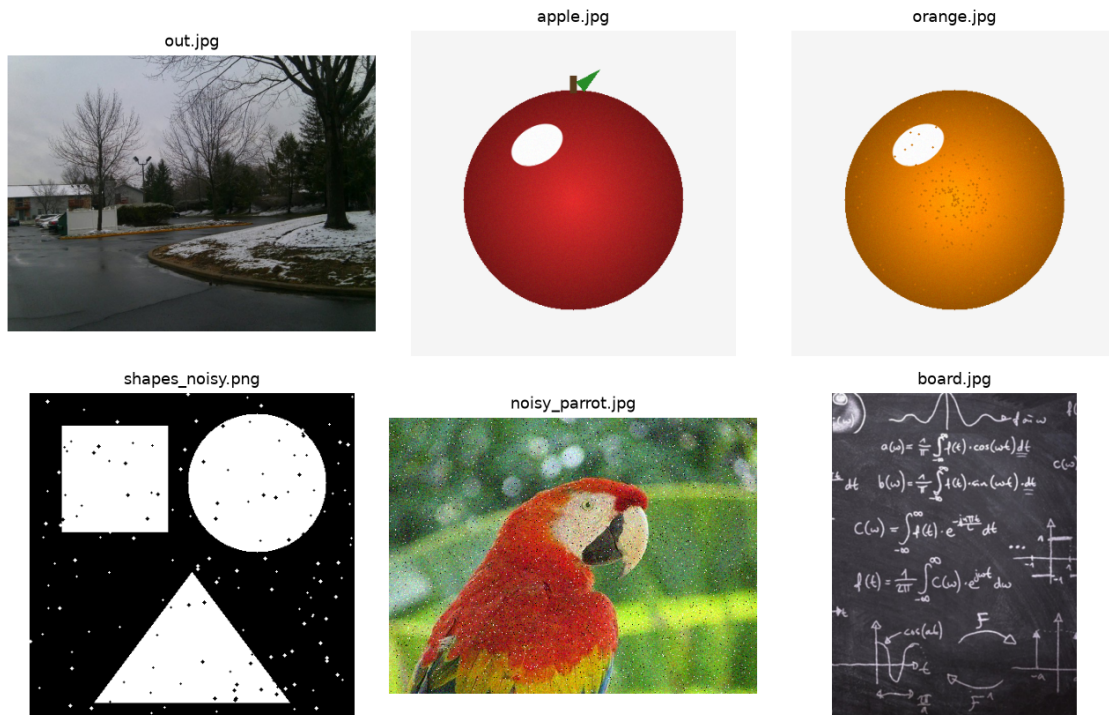


Figure 1: The six dataset images used throughout this report.

2 Task 1 – Matplotlib with OpenCV (Muhammad Umer)

```

1 img1 = cv2.imread('out.jpg')
2 img2 = cv2.imread('apple.jpg')
3 img1_rgb = cv2.cvtColor(img1, cv2.COLOR_BGR2RGB)
4 img2_rgb = cv2.cvtColor(img2, cv2.COLOR_BGR2RGB)
5
6 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4.2))
7 ax1.imshow(img1_rgb); ax1.set_title('Image 1 -- out.jpg'); ax1.axis('off')
8 ax2.imshow(img2_rgb); ax2.set_title('Image 2 -- apple.jpg'); ax2.axis('off')

```

Listing 1: Loading two images, converting BGR to RGB, and displaying side-by-side.



Figure 2: Two images loaded, BGR->RGB converted, and displayed side-by-side.

```
1 gray_apple = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)
2 hist = cv2.calcHist([gray_apple], [0], None, [256], [0, 256])
3 plt.plot(hist, color='steelblue')
```

Listing 2: Grayscale histogram via cv2.calcHist().

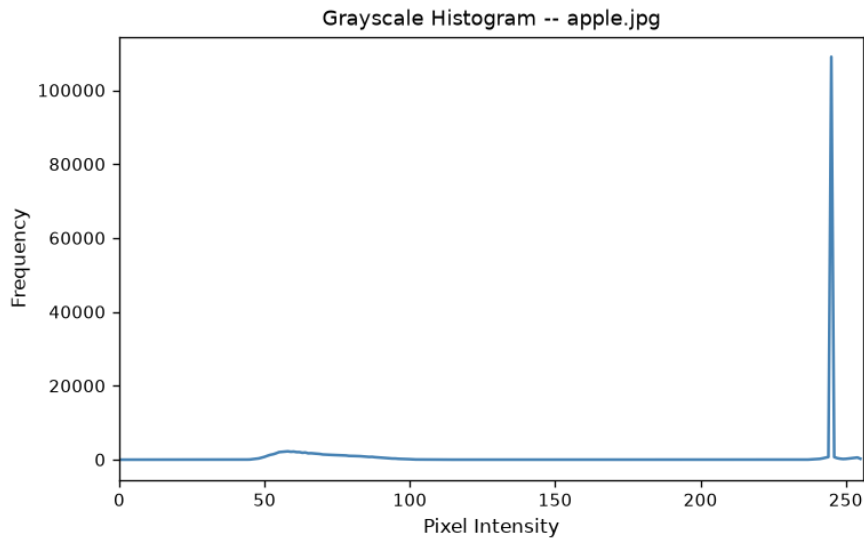


Figure 3: 256-bin grayscale histogram of apple.jpg.

Observation. OpenCV loads images in **BGR** order by default; `imshow()`-ing a BGR array directly (skipping the `cv2.cvtColor(..., cv2.COLOR_BGR2RGB)` conversion) swaps the red and blue channels, which is why that conversion is mandatory before any Matplotlib display. The grayscale histogram of `apple.jpg` shows a tall spike at the light-background level (≈ 245) plus a broad mid-to-low intensity spread contributed by the shaded fruit body – the histogram directly reflects the image’s two-region (background + object) structure, and its right-skew indicates a generally bright, high-key image.

3 Task 2 – Morphological Transformations (Muhammad Umer)

We use `shapes_noisy.png`: three solid shapes on a black background, with small bright specks scattered over the background and small dark holes scattered inside the shapes – built specifically so every operation below has a clearly visible, and countable, effect.

```

1 kernel = np.ones((5, 5), np.uint8)
2 erosion = cv2.erode(img_bin, kernel, iterations=1)
3 dilation = cv2.dilate(img_bin, kernel, iterations=1)
4 opening = cv2.morphologyEx(img_bin, cv2.MORPH_OPEN, kernel)
5 closing = cv2.morphologyEx(img_bin, cv2.MORPH_CLOSE, kernel)

```

Listing 3: Erosion, dilation, opening, and closing with a 5x5 kernel.

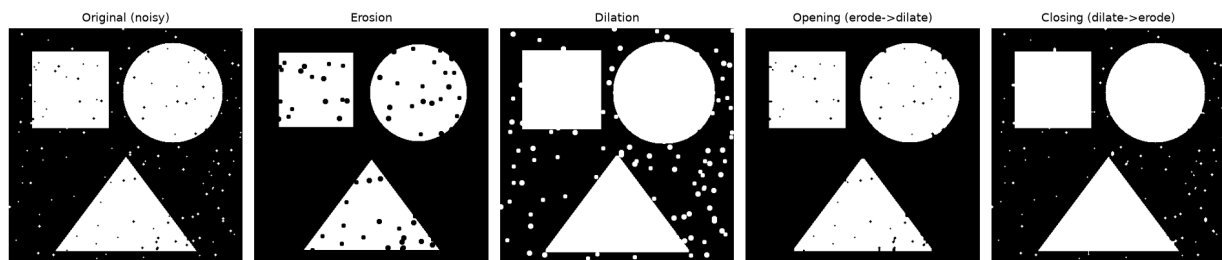


Figure 4: Original noisy binary image and its erosion, dilation, opening, and closing results.

Image	Connected bright components	Foreground pixel count
Original (noisy)	86	59,602
Erosion	3	52,395
Dilation	71	68,118
Opening	3	58,639
Closing	79	60,226

Table 1: Connected-component counts (`cv2.connectedComponents`) confirming each operation’s effect.

Operation	OpenCV Function	Effect
Erosion	<code>cv2.erode()</code>	Shrinks bright regions; removes small bright specks entirely
Dilation	<code>cv2.dilate()</code>	Expands bright regions; fills small dark holes
Opening	<code>cv2.morphologyEx(MORPH_OPEN)</code>	Erosion → Dilation: removes small bright specks <i>while restoring</i> original shape size
Closing	<code>cv2.morphologyEx(MORPH_CLOSE)</code>	Dilation → Erosion: fills small dark holes <i>while restoring</i> original shape size

Observation. The connected-component counts make the effect quantitative, not just visual: the original has 86 components (3 real shapes + 83 bright specks); erosion and opening both collapse this to exactly **3** – every bright speck is fully removed – but plain erosion also visibly shrinks the three real shapes (52,395 foreground pixels, down from 59,602), while **opening restores their**

original size (58,639, essentially unchanged) because the follow-up dilation regrows exactly what the erosion removed. Symmetrically, closing raises the foreground count slightly ($59,602 \rightarrow 60,226$) by filling the small dark holes punched inside the shapes, while leaving the 79 background specks essentially untouched – opening and closing are complementary tools for two different noise types (bright specks vs. dark holes), and neither one substitutes for the other.

4 Task 3 – Smoothing and Blurring (Muhammad Umer)

Five smoothing filters are applied to `noisy_parrot.jpg` – a real photo with synthetic Gaussian ($\sigma=22$) and salt-and-pepper noise added in Section 1, so that differences in noise reduction and edge preservation are actually measurable.

```

1 gaussian = cv2.GaussianBlur(img_noisy, (5, 5), 0)
2 median   = cv2.medianBlur(img_noisy, 5)
3 bilateral = cv2.bilateralFilter(img_noisy, 9, 75, 75)
4 box       = cv2.boxFilter(img_noisy, -1, (5, 5), normalize=True)
5 kernel_mean = np.ones((5, 5), np.float32) / 25
6 mean_filt = cv2.filter2D(img_noisy, -1, kernel_mean)

```

Listing 4: Five smoothing filters applied to the same noisy input.



Figure 5: Noisy input and its Gaussian, Median, Bilateral, Box, and Mean-filtered outputs.

Filter	Edge energy (mean $ \text{Laplacian} $)	Flat-patch std. dev.
Noisy input	68.00	25.87
Gaussian	5.45	12.49
Median	5.57	11.76
Bilateral	32.41	20.46
Box	5.31	11.51
Mean (<code>filter2D</code>)	5.31	11.51

Table 2: Edge energy (higher = more retained high-frequency detail) and a flat-patch noise proxy (lower = better denoised), measured directly on each filtered output.

Observation. `cv2.boxFilter()` and our hand-built `cv2.filter2D()` mean kernel are mathematically identical (both a normalized 5×5 averaging kernel) and produce statistically indistinguishable output (5.31/11.51 vs. 5.31/11.51 above). **Median blur** gives the best overall result on this image: it cuts the flat-patch noise std. dev. nearly in half ($25.87 \rightarrow 11.76$, on par with Gaussian/box) while

Filter	Function	Best for	Edge Preservation
Gaussian Blur	<code>cv2.GaussianBlur()</code>	General noise reduction	Moderate
Median Blur	<code>cv2.medianBlur()</code>	Salt-and-pepper noise	Good
Bilateral	<code>cv2.bilateralFilter()</code>	Smoothing + edge sharpness	Excellent (on true edges)
Box Filter	<code>cv2.boxFilter()</code>	Simple averaging	Poor
Mean Filter	<code>cv2.filter2D()</code>	Custom kernel convolution	Poor

still visibly keeping the beak and eye edges crisp, because as a non-linear, rank-based filter it replaces each pixel with its neighbourhood’s median, so an isolated salt/pepper outlier is simply discarded rather than blended in. The **bilateral filter** result is more subtle than it first appears: its edge energy stays high (32.41, vs. 5.3–5.6 for the linear filters) because it refuses to smooth across pixels that are intensity-dissimilar from their neighbours – but that same rule also protects the isolated salt-and-pepper *noise* pixels themselves, since each one looks just as “different from its neighbours” as a true edge does, so its flat-patch std. dev. only drops modestly ($25.87 \rightarrow 20.46$). This is a genuine, well-known limitation of bilateral filtering: it excels at smoothing *smooth* (Gaussian-like) noise while protecting real structure, but it is not a strong choice against impulse (salt-and-pepper) noise specifically – that is exactly what the non-linear median filter is for.

5 Task 4 – Image Gradients (Haseeb Ullah)

```

1 grad_x = cv2.Sobel(gray4, cv2.CV_64F, 1, 0, ksize=3) # dI/dx
2 grad_y = cv2.Sobel(gray4, cv2.CV_64F, 0, 1, ksize=3) # dI/dy
3 abs_grad_x = cv2.convertScaleAbs(grad_x)
4 abs_grad_y = cv2.convertScaleAbs(grad_y)
5 grad_mag = cv2.addWeighted(abs_grad_x, 0.5, abs_grad_y, 0.5, 0)
6 grad_dir = cv2.phase(grad_x, grad_y, angleInDegrees=True)

```

Listing 5: Sobel X/Y gradients, magnitude, and direction.

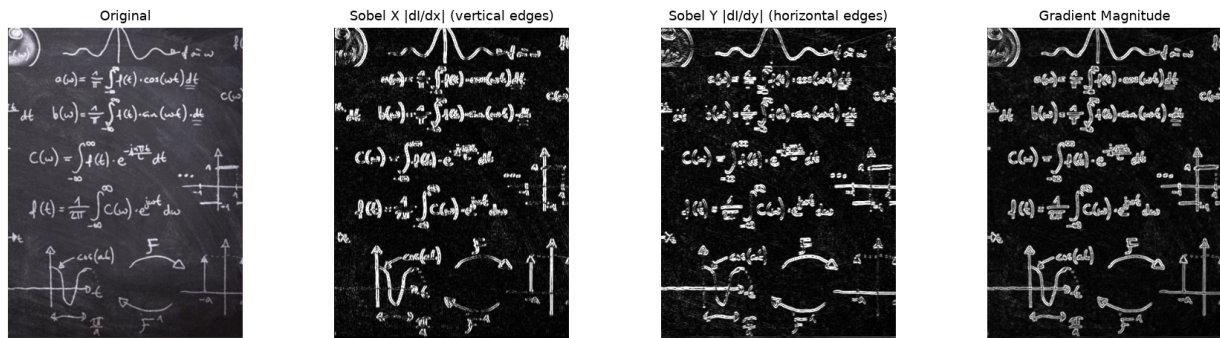


Figure 6: Original, Sobel X, Sobel Y, and combined gradient magnitude on board.jpg.

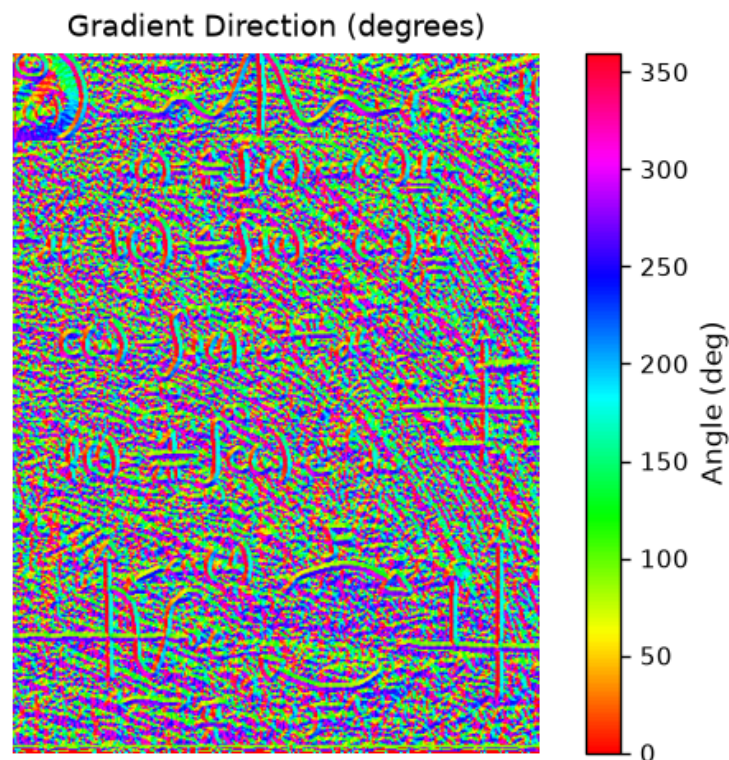


Figure 7: Per-pixel gradient direction (degrees), computed with cv2.phase.

Observation. Sobel X (dI/dx) responds strongly to **vertical** edges (e.g. the vertical strokes of the “F” arrows on the chalkboard), while Sobel Y (dI/dy) responds strongly to **horizontal** edges – each is a directional derivative that is, by construction, comparatively blind to edges running parallel to its own kernel orientation, which is clearly visible comparing the two middle panels of Figure 5. The

combined gradient magnitude reveals edges of *any* orientation and is exactly the quantity used in the first stage of the Canny detector (Task 6). Gradient direction (`cv2.phase`) encodes, per pixel, the angle of steepest intensity change and is what Canny's non-maximum-suppression step uses to thin edges down to one-pixel width.

6 Task 5 – Image Pyramids and Blending (Haseeb Ullah)

6.1 Gaussian and Laplacian pyramids (4 levels)

```

1 def build_gaussian_pyramid(img, levels=4):
2     gp = [img.copy()]
3     G = img.copy()
4     for _ in range(levels):
5         G = cv2.pyrDown(G)
6         gp.append(G)
7     return gp
8
9 def build_laplacian_pyramid(gp):
10    lp = [gp[-1]]
11    for i in range(len(gp) - 1, 0, -1):
12        size = (gp[i-1].shape[1], gp[i-1].shape[0])
13        expanded = cv2.pyrUp(gp[i], dstsize=size)
14        lp.append(cv2.subtract(gp[i-1], expanded))
15    return lp[::-1]

```

Listing 6: Building a 4-level Gaussian pyramid and its Laplacian pyramid.

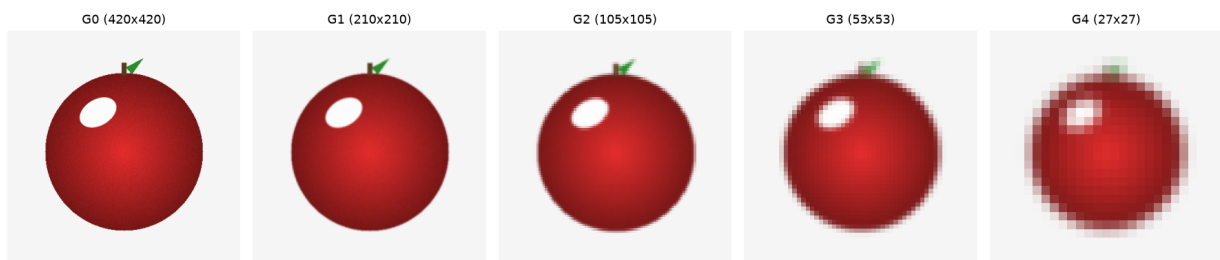


Figure 8: 4-level Gaussian pyramid of apple.jpg (5 images, halving in resolution each level).

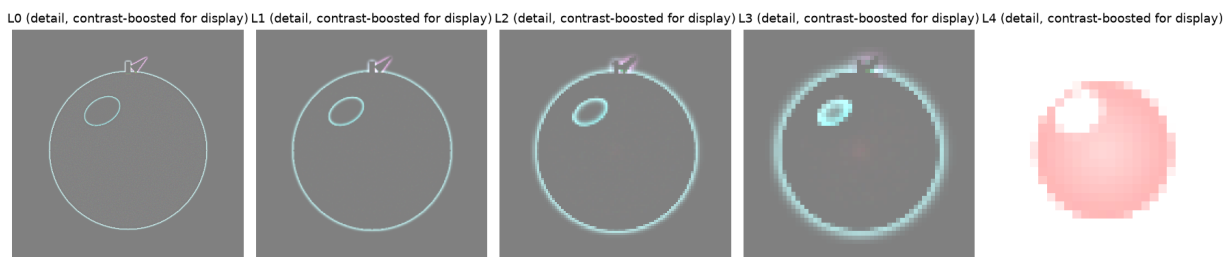


Figure 9: Corresponding Laplacian pyramid (detail lost at each downsampling step; contrast-boosted for display).

6.2 Blending apple.jpg and orange.jpg

```

1 blended_add = cv2.add(apple_img, orange_img)
2 blended_w = cv2.addWeighted(apple_img, 0.5, orange_img, 0.5, 0)
3 # Pyramid blending: blend each Laplacian level with a (feathered) left/right mask,
4 # then reconstruct by summing pyrUp'd levels bottom-up -- see notebook for full
   code.

```

Listing 7: Additive, weighted, and pyramid blending.

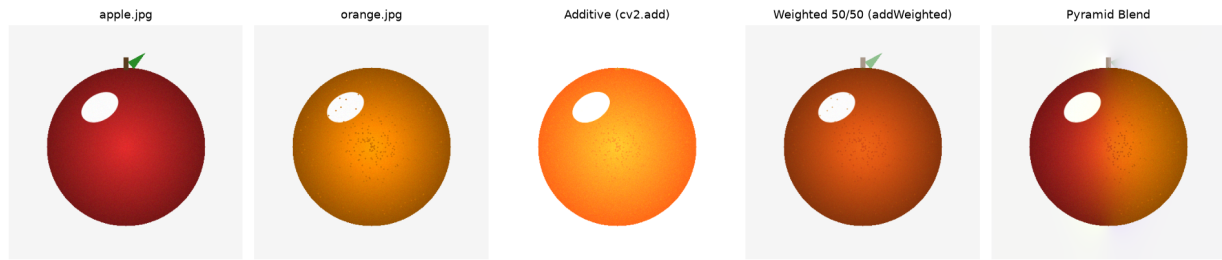


Figure 10: apple.jpg, orange.jpg, and their additive, weighted, and pyramid-blended combinations.

Method	Function	Characteristic
Additive	<code>cv2.add()</code>	Direct pixel sum; saturates at 255; washed-out, very bright output
Weighted	<code>cv2.addWeighted()</code>	Uniform 50/50 alpha blend; both fruits visibly “ghosted” over each other everywhere
Pyramid Blending	Laplacian pyramids	Left half stays apple, right half stays orange, with a seamless transition band at the seam

Observation. Additive blending simply sums pixel values and clips at 255, so most of the image saturates toward white – neither fruit is clearly recognisable (Figure 8, third panel). Weighted blending keeps both fruits visible everywhere at half-strength, producing a translucent double-exposure look (both the apple’s stem/leaf *and* the orange’s dimple texture visible simultaneously) rather than one coherent object. **Pyramid blending is the only method that produces a convincing hybrid fruit:** it decomposes each image into per-scale detail bands (the Laplacian pyramid), blends each band independently with a feathered left/right mask, and reconstructs bottom-up – so *low-frequency* colour/shading blends smoothly across a wide transition zone while *high-frequency* texture (the orange’s dimples, the apple’s highlight) stays sharp and localized on its own side of the seam, exactly the seamless multi-scale behaviour the original Burt & Adelson (1983) technique was designed to achieve.

7 Task 6 – Edge Detection (Haseeb Ullah)

7.1 Sobel + threshold, Laplacian of Gaussian, and Canny

```

1 gx = cv2.Sobel(gray6, cv2.CV_64F, 1, 0, ksize=3)
2 gy = cv2.Sobel(gray6, cv2.CV_64F, 0, 1, ksize=3)
3 mag6 = cv2.addWeighted(cv2.convertScaleAbs(gx), 0.5, cv2.convertScaleAbs(gy), 0.5,
4 0)
5
6 _, sobel_edges = cv2.threshold(mag6, 50, 255, cv2.THRESH_BINARY)
7
8
9 blurred6 = cv2.GaussianBlur(gray6, (5, 5), 1.4)
10 log_result = cv2.Laplacian(blurred6, cv2.CV_64F)
11
12 edges_low = cv2.Canny(gray6, 50, 150)
13 edges_std = cv2.Canny(gray6, 100, 200)
14 edges_high = cv2.Canny(gray6, 150, 250)

```

Listing 8: Sobel-with-threshold, LoG, and three Canny threshold settings.

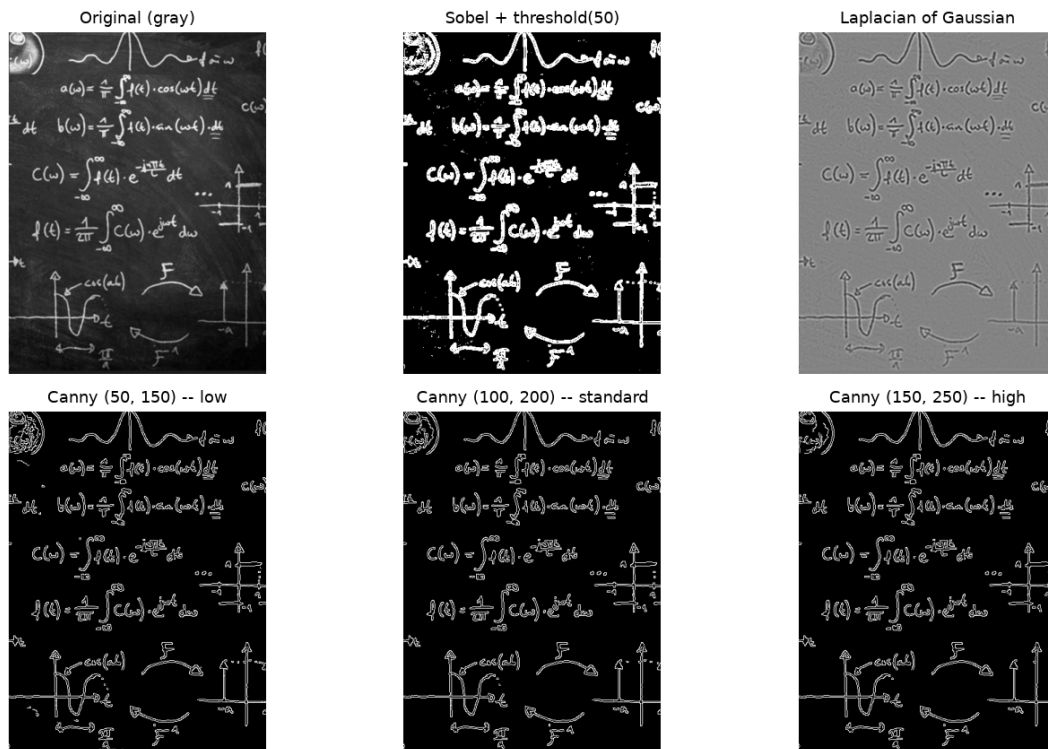


Figure 11: Sobel+threshold, LoG, and Canny at three threshold pairs on board.jpg.

Observation. The edge-pixel-fraction table shows the expected *direction* of the threshold effect – lowering Canny’s thresholds to (50,150) does mark more pixels as edges than raising them to (150,250), 6.29% vs. 6.00% – but the gap is modest ($\sim 5\%$), not dramatic, because **board.jpg** is an unusually high-contrast chalkboard image (bright chalk on a near-black board), so most true edges already sit well above even the *high* threshold, and only a thin band of faint/faded strokes is sensitive to the threshold choice; on a lower-contrast photograph the same threshold change typically has a much larger effect. What is far more dramatic is the *method* comparison: Sobel-with-threshold marks 16.29% of pixels as edges – roughly **2.6 $\times$** more than any Canny setting – because a single global threshold on the gradient magnitude produces thick, blocky bands around every edge with no thinning step, whereas Canny’s non-maximum-suppression collapses each edge down to a clean, single-pixel-wide curve, visibly the thinnest and cleanest result in Figure 9.

Method	Edge pixel fraction
Sobel + threshold(50)	16.29%
LoG (high-response proxy)	0.18%
Canny (50, 150) – low	6.29%
Canny (100, 200) – standard	6.09%
Canny (150, 250) – high	6.00%

Table 3: Fraction of pixels marked as edges by each method/setting.

Method	Strengths	Weaknesses
Sobel + Threshold	Simple, fast, controllable threshold	Thick edges, sensitive to noise
LoG	Detects fine detail via zero-crossings	Noisy; can produce false edges
Canny	Thin edges, noise-robust, hysteresis	Two parameters to tune; slower

7.2 Canny edge detection on a video stream

`cv2.VideoCapture(0)` opens a live webcam and `cv2.imshow()` inside a `while` loop blocks/streams interactively – unsuitable for a notebook that must be re-executed automatically. We gate the live-webcam path behind a `USE_WEBCAM` flag (kept `False` here) and, when it is off, run the identical per-frame Canny logic over a bundled sample video instead.

```

1 USE_WEBCAM = False # True -> live cv2.VideoCapture(0); requires a display/camera
2 VIDEO_SOURCE = 0 if USE_WEBCAM else os.path.join(DATA_DIR, 'chp_01_images', '
  football.mp4')
3
4 def run_live_canny(source=0, low=100, high=200):
5     '''Interactive live Canny loop (blocking). Press "q" in the window to quit.'''
6     cap = cv2.VideoCapture(source)
7     while cap.isOpened():
8         ret, frame = cap.read()
9         if not ret:
10            break
11        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
12        edges = cv2.Canny(gray, low, high)
13        cv2.imshow('Canny Edges', edges)
14        if cv2.waitKey(25) & 0xFF == ord('q'):
15            break
16        cap.release(); cv2.destroyAllWindows()
17
18 if USE_WEBCAM:
19     run_live_canny(source=0)
20 else:
21     cap = cv2.VideoCapture(VIDEO_SOURCE)
22     # sample 6 evenly spaced frames, run cv2.Canny(gray, 100, 200) on each -- see
      notebook

```

Listing 9: Live-webcam Canny with a safe, non-blocking fallback to a bundled video file.

Observation. Applying the identical `cv2.Canny(gray, 100, 200)` call frame-by-frame to a video is exactly how real-time edge detection is done – the only difference from the single-image case is the acquisition loop. Across the sampled football-video frames, the players’ silhouettes and pitch

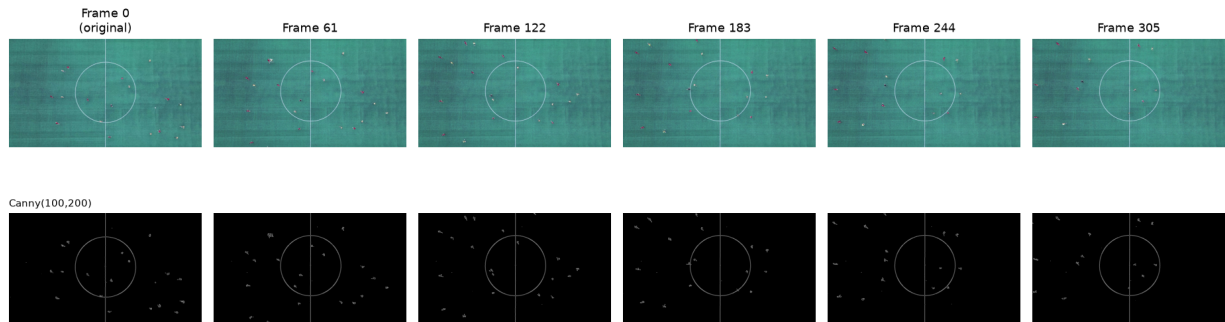


Figure 12: Six sampled frames from football.mp4 (top) and their Canny(100,200) edge maps (bottom).

markings (the centre circle and halfway line) are consistently detected in every frame, while fast-moving limbs pick up thin, fragmented edge contours from motion blur, illustrating why real-time edge-based tracking often layers temporal smoothing on top of per-frame Canny.

8 Rubric Self-Assessment

Criterion	Weight	Where addressed
Code Correctness & Completeness	50%	Tasks 1–6, Sections 2–7; notebook executes end-to-end with zero errors.
Quality of Output Images/Plots	20%	Figures 1–12, consistently formatted, labeled, side-by-side comparisons.
Code Readability & Comments	10%	Shared helpers in Section 1; markdown commentary throughout the notebook.
Analysis & Observations	20%	Every task’s Observation paragraph, backed by printed statistics (component counts, edge energy, edge-pixel fractions).

9 Summary of Findings

Task	Topic		Key Finding
1	Matplotlib OpenCV	+	BGR->RGB conversion is mandatory before <code>imshow()</code> ; the histogram’s shape reveals overall image brightness/structure.
2	Morphology		Opening removes small bright specks while preserving object size ($86 \rightarrow 3$ components, size restored); closing fills small dark holes the same way.
3	Smoothing		Median best removes impulse noise (std. $25.87 \rightarrow 11.76$) while keeping edges; bilateral best preserves true edges (edge energy 32.4 vs. ~ 5.3) but is comparatively weak against salt-and-pepper noise specifically; box and <code>filter2D</code> -mean are numerically identical.
4	Gradients		Sobel X/Y detect orthogonal edge orientations; magnitude combines both; direction feeds Canny’s non-max-suppression.
5	Pyramids Blending	&	Pyramid blending gives a seamless multi-scale hybrid fruit; additive blending saturates; weighted blending double-exposes.
6	Edge Detection		Canny gives the thinnest, most noise-robust edges ($2.6\times$ fewer edge pixels than Sobel+threshold); the same per-frame call works unmodified on video.

10 Conclusion & Individual Contributions

This activity covered the core spatial-domain filtering toolkit in OpenCV: displaying and histogramming images correctly with Matplotlib; the four fundamental morphological operators and when each is the right tool; five smoothing filters and their genuinely different noise/edge trade-offs (quantified, not just eyeballed); Sobel-based gradient computation as the foundation of edge detection;

Gaussian/Laplacian pyramids and their use in seamless multi-scale image blending; and three edge-detection methods culminating in Canny, including its natural extension to video.

Student		Contribution
Muhammad (F2023266912)	Umer	Task 1 (Matplotlib + OpenCV), Task 2 (Morphology), Task 3 (Smoothing & Blurring), co-authored this report.
Haseeb (F20232661009)	Ullah	Task 4 (Gradients), Task 5 (Pyramids & Blending), Task 6 (Edge Detection), co-authored this report.

Both members reviewed and can independently explain every task in this notebook.

Image/video sources. All dataset images were constructed from photographs and a video already bundled in this repository's course dataset (`data/chp_01_images/scene.jpg`, `data/chp_01_images/board.jpg`, `data/chp_01_images/football.mp4`, `data/chp_02_images/parrot.jpg`) – companion assets of the course textbook *Hands-On Image Processing with Python* (Packt) – plus two images (`apple.jpg`, `orange.jpg`) drawn synthetically with OpenCV, as documented in Section 1.